



IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Cluster analysis: Part - III

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Clustering analysis part III



Agenda

- Handling missing data
- Calculation of similarity and dissimilarity matrix

Handling missing data

- It often happens that not all measurements are actually available, so there are some “holes” in the data matrix
- Such an absent measurement is called a missing value and it may have several causes
- The value of the measurement may have been lost or it may not have been recorded at all by oversight or lack of time

Handling missing data

- Sometimes the information is simply not available, for example the birthdate of a foundling, or the patient may not remember whether he or she ever had the measles, or it may be impossible to measure the desired quantity due to the malfunctioning of some instrument
- In certain instances the question does not apply (such as the colour of hair of a bald person) or there may be more than one possible answer (when two experimenters obtain very different results)

Handling missing data

- How can we handle a data set with missing values?
- In a matrix we indicate the absent measurements by means of some code
- If there exists an object in the data set for which all measurements are missing, there is really no information on this object so it has to be deleted
- Analogously, a variable consisting exclusively of missing values has to be removed too

Handling missing data

- If the data are standardized, the mean value m_f of the f^{th} variable is calculated by making use of the present values only
- The same goes for s_f ,

$$s_f = \frac{1}{n} \{ |x_{1f} - m_f| + |x_{2f} - m_f| + \cdots + |x_{nf} - m_f| \}$$

In the denominator , we must replace 'n' by the number of non missing values for that variable

- But of course only when the corresponding x_i is not missing itself

Handling missing data

- In the computation of distances (based on either the x_i , or the z_i ,) similar precautions must be taken
- When calculating the distances $d(i, j)$, only those variables are considered in the sum for which the measurements for both objects are present subsequently the sum is multiplied by p and divided by the actual number of terms (in the case of Euclidean distances this is done before taking the square root)
- Such a procedure only makes sense when the variables are thought of as having the same weight (for instance, this can be done after standardization)

Handling missing data

- When computing these distances, one might come across a pair of objects that do not have any common measured variables, so their distance cannot be computed by means of the above mentioned approach.
- Several remedies are possible: One could remove either object or one could fill in some average distance value based on the rest of the data
- Or by replacing all missing x_{if} by the mean m_f of that variable; then all distances can be computed
- Applying any of these methods, one finally possesses a “full” set of distances

Dissimilarities

- The entries of a n -by n matrix may be Euclidean or Manhattan distances
- However, there are many other possibilities, so we no longer speak of distances but of dissimilarities (or dissimilarity coefficients)
- Basically, dissimilarities are non-negative numbers $d(i, j)$ that are small (close to zero) when i and j are “near” to each other and that become large when i and j are very different
- We shall usually assume that dissimilarities are symmetric and that the dissimilarity of an object to itself is zero, but in general the triangle inequality does not hold

Dissimilarities

- Dissimilarities can be obtained in several ways.
- Often they can be computed from variables that are binary, nominal, ordinal, interval, or a combination of these
- Also, dissimilarities can be simple subjective ratings of how much certain objects differ from each other, from the point of view of one or more observers
- This kind of data is typical in the social sciences and in marketing

Example

- Fourteen postgraduate economics students (coming from different parts of the world) were asked to indicate the subjective dissimilarities between 11 scientific disciplines.
- All of them had to fill in a matrix like Table 4, where the dissimilarities had to be given as integer numbers on a scale from 0 (identical) to 10 (very different)
- The actual entries of the Table in next slide, are the averages of the values given by the students

Example

- It appears that the smallest dissimilarity is perceived between mathematics and computer science (1.43), whereas the most remote fields were psychology and astronomy (9.36)

Astronomy	0.00											
Biology	7.86	0.00										
Chemistry	6.50	2.93	0.00									
Computer sci.	5.00	6.86	6.50	0.00								
Economics	8.00	8.14	8.21	4.79	0.00							
Geography	4.29	7.00	7.64	7.71	5.93	0.00						
History	8.07	8.14	8.71	8.57	5.86	3.86	0.00					
Mathematics	3.64	7.14	4.43	1.43	3.57	7.07	9.07	0.00				
Medicine	8.21	2.50	2.93	6.36	8.43	7.86	8.43	6.29	0.00			
Physics	2.71	5.21	4.57	4.21	8.36	7.29	8.64	2.21	5.07	0.00		
Psychology	9.36	5.57	7.29	7.21	6.86	8.29	7.64	8.71	3.79	8.64	0.00	

Dissimilarities

- If one wants to perform a cluster analysis on a set of variables that have been observed in some population, there are other measures of dissimilarity
- For instance, one can compute the (parametric) Pearson product-moment between the variables f and g , or alternatively the (non-parametric) Spearman correlation

Dissimilarities

- Both coefficients lie between - 1 and + 1 and do not depend on the choice of measurement units
- The main difference between them is that the Pearson coefficient looks for a linear relation between the variables f and g, whereas the Spearman coefficient searches for a monotone relation

$$R(f, g) = \frac{\sum_{i=1}^n (x_{if} - m_f)(x_{ig} - m_g)}{\sqrt{\sum_{i=1}^n (x_{if} - m_f)^2} \sqrt{\sum_{i=1}^n (x_{ig} - m_g)^2}}$$

$$\rho_{(x,y)} = \frac{\text{Cov}(x,y)}{\sigma_x \cdot \sigma_y}$$

Dissimilarities

- Correlation coefficients are useful for clustering purposes because they measure the extent to which two variables are related
- Correlation coefficients, whether parametric or nonparametric, can be converted to dissimilarities $d(f, g)$, for instance by setting

$$\underline{d(f, g) = (1 - R(f, g))/2}$$

With this formula, variables with a high positive correlation receive a dissimilarity coefficient close to zero, whereas variables with a strongly negative correlation will be considered very dissimilar

Similarities

- The more objects i and j are alike (or close), the larger $s(i, j)$ becomes
- Such a similarity $s(i, j)$ typically takes on values between 0 and 1, where 0 means that i and j are not similar at all and 1 reflects maximal similarity
- Values in between 0 and 1 indicate various degrees of resemblance
- Often it is assumed that the following conditions hold:

$$(S1) \quad 0 \leq s(i, j) \leq 1 \quad \checkmark$$

$$(S2) \quad s(i, i) = 1 \quad \checkmark$$

$$(S3) \quad s(i, j) = s(j, i) \quad \checkmark$$

Similarities

- For all objects i and j , the numbers $s(i, j)$ can be arranged in an n -by- n matrix, which is then called a similarity matrix
- Both similarity and dissimilarity matrices are generally referred to as proximity matrices, or sometimes as resemblance
- In order to define similarities between variables, we can again resort to the Pearson or the Spearman correlation coefficient
- However, neither correlation measure can be used directly as a similarity coefficient because they also take on negative values

Similarities

- Some transformation is in order to bring the coefficients into the zero-one range
- There are essentially two ways to do this, depending on the meaning of the data and the purpose of the application
- If variables with a strong negative correlation are considered to be very different because they are oriented in the opposite direction (like mileage and weight of a set of cars), then it is best to take something like the following:

$$\underline{s(f, g) = (1 + R(f, g))/2}$$

which yields $s(f, g) = 0$ whenever $R(f, g) = -1$.

Similarities

- There are situations in which variables with a strong negative correlation should be grouped, because they measure essentially the same thing
- For instance, this happens if one wants to reduce the number of variables in a regression data set by selecting one variable from each cluster
- In that case it is better to use a formula like

$$\underline{s(f, g) = |R(f, g)|}$$

which yields $s(f, g) = 1$ when $\underline{R(f, g) = -1}$

Similarities

- Suppose the data consist of a similarity matrix but one wants to apply a clustering algorithm designed for dissimilarities
- Then it is necessary to transform the similarities into dissimilarities
- The larger the similarity $s(i, j)$ between i and j , the smaller their dissimilarity $d(i, j)$ should be
- Therefore, we need a decreasing transformation, such as

$$\underline{d(i, j) = 1 - s(i, j)}$$

Binary Variables

- A contingency table for binary variables.

		object j		
		1	0	sum
object i	1	<u>q</u>	r	$q + r$
	0	s	<u>t</u>	<u>$s + t$</u>
	sum	$q + s$	$r + t$	p

Dissimilarity between two binary variables

- $q \rightarrow$ is the number of variables that equal 1 for both objects i and j ,
- $r \rightarrow$ is the number of variables that equal 1 for object i but that are 0 for object j ,
- $s \rightarrow$ is the number of variables that equal 0 for object i but equal 1 for object j , and
- $t \rightarrow$ is the number of variables that equal 0 for both objects i and j .
- The total number of variables is p , where $p = q+r+s+t$.

Symmetric Binary Dissimilarity

$$d(i, j) = \frac{r + s}{q + r + s + t}.$$

Gender

0 - male

1 - female

Asymmetric binary variable

- A binary variable is asymmetric if the outcomes of the states are not equally important, such as the *positive* and *negative* outcomes of a disease *test*.
- By convention, we shall code the most important outcome, which is usually the rarest one, by 1 (e.g., *HIV positive*) and the other by 0 (e.g., *HIV negative*).
- Given two asymmetric binary variables, the agreement of two 1s (a positive match) is then considered more significant than that of two 0s (a negative match).
- Therefore, such binary variables are often considered “monary” (as if having one state).

asymmetric binary dissimilarity

		object j		
		1	0	sum
object i	1	q	r	$q + r$
	0	s	t	$s + t$
	sum	$q + s$	$r + t$	p

$$d(i, j) = \frac{r + s}{q + r + s}.$$

Jaccard coefficient

$$\text{sim}(i, j) = \frac{q}{q + r + s} = 1 - d(i, j).$$

Dissimilarity between binary variables

<i>name</i>	<i>gender</i>	<i>fever</i>	<i>cough</i>	<i>test-1</i>	<i>test-2</i>	<i>test-3</i>	<i>test-4</i>
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	Y	N	N	N	N
:	:	:	:	:	:	:	:

Dissimilarity between Jack and Marry

Jack

name	gender	fever	cough	test-1	test-2	test-3	test-4
Jack	M	Y	N	P	N	N	N
Marry	F	Y	N	P	N	P	N
Jim	M	Y	Y	N	N	N	N
:	:	:	:	:	:	:	:

Marry

	1	0
1	2	1
0	0	3

$$d(\text{Jack}, \text{Marry}) = \frac{0+1}{2+0+1} = 0.33$$

Dissimilarity between Jack and Jim

Jim

name	gender	fever	cough	test-1	test-2	test-3	test-4
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	Y	N	N	N	N
:	:	:	:	:	:	:	:

Jack

	1	0
1	1	1
0	1	3

$$d(\text{Jack}, \text{Jim}) = \frac{1+1}{1+1+1} = 0.67$$

Dissimilarity between Jim and Marry

Jim

name	gender	fever	cough	test-1	test-2	test-3	test-4
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	Y	N	N	N	N
:	:	:	:	:	:	:	:

Marry

	1	0
1	1	2
0	1	2

$$d(\text{Mary}, \text{Jim}) = \frac{1+2}{1+1+2} = 0.75$$

Thank you





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Cluster analysis: Part - IV

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

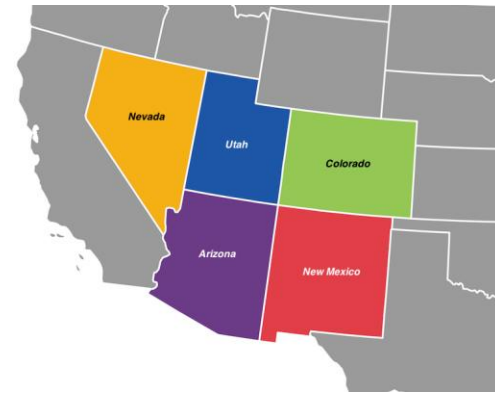
- How to handle the following types of variables :
 - Interval scale variable
 - Binary variables
 - Categorical Variables
 - Ordinal Variables
 - Ratio-Scaled Variables
 - Variables of mixed type



Categorical Variables

Categorical Variables

- A categorical variable is a generalization of the binary variable in that it can take on more than two states
- For example, map color is a categorical variable that may have, say, five states: red, yellow, green, purple, and blue



Categorical Variables

- Let the number of states of a categorical variable be M
- The states can be denoted by letters, symbols, or a set of integers, such as $1, 2, \dots, M$
- Notice that such integers are used just for data handling and do not represent any specific ordering



Categorical Variables

- “How is dissimilarity computed between objects described by categorical variables?”



Categorical Variables

- The dissimilarity between two objects i and j can be computed based on the ratio of mismatches:
$$d(i, j) = \frac{p - m}{p},$$

where 'm' is the number of matches (i.e., the number of variables for which 'i' and 'j' are in the same state), and 'p' is the total number of variables

Weights can be assigned to increase the effect of 'm' or to assign greater weight to the matches in variables having a larger number of states

Dissimilarity between categorical variables

- Suppose that we have the sample data as shown in the table
- Let only the object-identifier and the variable (or attribute) test-1 are available, which is a categorical data

object identifier	test-1 (categorical)	test-2 (ordinal)	test-3 (ratio-scaled)
1	code-A	excellent	445
2	code-B	fair	22
3	code-C	good	164
4	code-A	excellent	1,210

Finding Groups in Data: An Introduction to Cluster Analysis

Author(s): [Leonard Kaufman](#), [Peter J. Rousseeuw](#)

March 1990, John Wiley & Sons, Inc.

Dissimilarity matrix

$$\begin{array}{c} 1 \quad 2 \quad 3 \quad 4 \\ \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} \left[\begin{array}{cccc} 0 & & & \\ \underline{d(2,1)} & 0 & & \\ \underline{d(3,1)} & \underline{d(3,2)} & 0 & \\ \underline{d(4,1)} & \underline{d(4,2)} & \underline{d(4,3)} & 0 \end{array} \right] \end{array}$$

Dissimilarity between categorical variables

- Since here we have one categorical variable, test-1, we set $p = 1$ in Equation
$$d(i, j) = \frac{p - m}{p},$$

So that $d(i, j)$ evaluates to '0' if objects i and j match, and '1' if the objects differ

- Thus, we get

$$\begin{bmatrix} 0 & & & \\ \boxed{1} & 0 & & \\ & 1 & 1 & 0 \\ \boxed{0} & 1 & 1 & 0 \end{bmatrix}$$

$$d(2,1) = (1-0)/1 = 1$$

$$d(4,1) = (1-1)/1 = 0$$

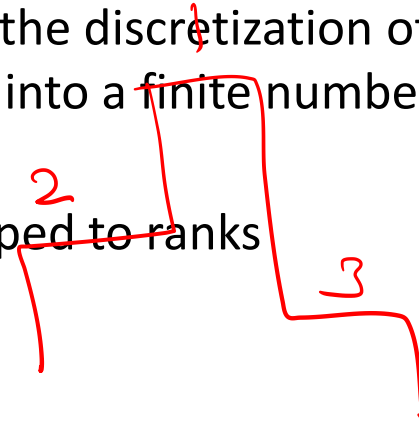
object identifier	test-1 (categorical)
1	code-A
2	code-B
3	code-C
4	code-A

Ordinal Variables

- A discrete ordinal variable resembles a categorical variable, except that the 'M' ~~states of the ordinal value~~ are ordered in a meaningful sequence
- Ordinal variables are ^{very useful for} registering subjective assessments of qualities that cannot be measured objectively
- For example, professional ranks are often enumerated in a sequential order, such as Assistant, Associate, and full for Professors
- A continuous ordinal variable looks like a set of continuous data of an unknown scale; that is, the relative ordering of the values ^{is essential} but their actual magnitude is not

Ordinal Variables

- For example, the relative ranking in a particular sport (e.g., gold, silver, bronze) is often more essential than the actual values of a particular measure
- Ordinal variables may also be obtained from the discretization of interval-scaled quantities by splitting the value range into a finite number of classes
- The values of an ordinal variable can be mapped to ranks



Dissimilarity computation

- The treatment of ordinal variables is quite similar to that of interval-scaled variables when computing the dissimilarity between objects
- Suppose that 'f' is a variable from a set of ordinal variables describing 'n' objects
- The dissimilarity computation with respect to 'f' involves the following steps:
→ →
- The value of 'f' for the i^{th} object is x_{if} , and 'f' has M_f ordered states, representing the ranking $1, \dots, M_f$.
- Replace each x_{if} by its corresponding rank, $r_{if} \in \{1, \dots, M_f\}$.

Dissimilarity computation

	A	B	C	D	E	F	G
B	69.8						
C	2.0	70.8					
D	71.6	5.7	72.5				
E	108.6	42.2	109.9	43.9			
F	95.7	26.3	96.8	26.4	19.1		
G	5.8	75.7	5.1	77.4	114.3	101.6	
H	17.7	87.2	17.3	89.2	125.0	112.9	12.2

Standardization of ordinal variable

- Since each ordinal variable can have a different number of states, it is often necessary to map ~~the~~ the range of each variable onto [0.0,1.0] so that each variable has equal weight.
- This can be achieved by replacing the rank r_{if} of the i^{th} object in the f^{th} variable by:

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

Dissimilarity computation

- Dissimilarity can then be computed using any of the distance measures described earlier (like that for interval data)

Example

- Suppose that we have the sample data of the following Table ,
- Except that this time only the object-identifier and the continuous ordinal variable, test-2, are available
- There are three states for test-2, namely fair, good, and excellent, that is $M_f = 3$

object identifier	test-1 (categorical)	test-2 (ordinal)	test-3 (ratio-scaled)
1	code-A	3	445
2	code-B	1	22
3	code-C	2	164
4	code-A	3	1,210

Example

- For step 1, if we replace each value for test-2 by its rank, the four objects are assigned the rank $z_{if} = \frac{r_{if} - 1}{M_f - 1}$, respectively
- Step 2 normalizes the ranking by mapping rank 1 to 0.0, rank 2 to 0.5, and rank 3 to 1.0
- For step 3, we can use, say, the Euclidean distance, which results in the following dissimilarity matrix:

—

Dissimilarity computation

Direct
indirect

$$1 \rightarrow 3 \rightarrow 1$$
$$2 \rightarrow 1 \rightarrow 0$$
$$3 \rightarrow 2 \rightarrow 0.5$$
$$4 \rightarrow 3 \rightarrow 1$$
$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & & \\ 1 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix} \end{matrix}$$

Ratio-Scaled Variables

- A ratio-scaled variable makes a positive measurement on a nonlinear scale, such as an exponential scale approximately following the formula

$$Ae^{Bt} \quad \text{or} \quad Ae^{-Bt}$$

where A and B are positive constants, and t typically represents time

- Common examples include the growth of a bacteria population or the decay of a radioactive element

Computing the dissimilarity between objects

- There are three methods to handle ratio-scaled variables for computing the dissimilarity between objects:
 1. Treat ratio-scaled variables like interval-scaled variables
 - This, however, is not usually a good choice since it is likely that the scale may be distorted ✓
 2. Apply logarithmic transformation to a ratio-scaled variable f having value x_{if} for object i by using the formula $y_{if} = \log(x_{if})$
 - The y_{if} values can be treated as interval valued, Notice that for some ratio-scaled variables, log-log or other transformations may be applied, depending on the variable's definition and the application

Computing the dissimilarity between objects

3. Treat x_{if} as continuous ordinal data and treat their ranks as interval-valued
- The latter two methods are the most effective, although the choice of method used may depend on the given application

Example

- This time, we have the sample data of the following Table,
- Except that only the object-identifier and the ratio-scaled variable, test-3, are available

object identifier	test-1 (categorical)	test-2 (ordinal)	test-3 (ratio-scaled)
1	code-A	excellent	445
2	code-B	fair	22
3	code-C	good	164
4	code-A	excellent	1,210

Example

- Let's try a logarithmic transformation ^{2 3 4}
- Taking the log of test-3 results in the values 2.65, 1.34, 2.21, and 3.08 for the objects 1 to 4, respectively
- Using the Euclidean distance on the transformed values, we obtain the following dissimilarity ^{2 3 4} matrix:

$$\begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} \begin{bmatrix} 0 & & & \\ 1.31 & 0 & & \\ 0.44 & 0.87 & 0 & \\ 0.43 & 1.74 & 0.87 & 0 \end{bmatrix}$$

object identifier (ratio-scaled)	test-3
1	445
2	22
3	164
4	1,210

Variables of Mixed Types

- So far we have discussed how to compute the dissimilarity between objects described by variables of the same type, where these types may be either interval-scaled, symmetric binary, asymmetric binary, categorical, ordinal, or ratio-scaled
- However, in many real databases, objects are described by a mixture of variable types

Variables of Mixed Types

- In general, a database can contain all of the six variable types listed above
- “So, how can we compute the dissimilarity between objects of mixed variable types?”
- One approach is to group each kind of variable together, performing a separate cluster analysis for each variable type
 - This is feasible if these analyses derive compatible results
 - However, in real applications, it is unlikely that a separate cluster analysis per variable type will generate compatible results

Variables of Mixed Types

- A more preferable approach is to process all variable types together, performing a single cluster analysis
- One such technique combines the different variables into a single dissimilarity matrix, bringing all of the meaningful variables onto a common scale of the interval $[0.0, 1.0]$

Variables of Mixed Types

- Suppose that the data set contains p variables of mixed type
- The dissimilarity $d(i, j)$ between objects i and j is defined as

$$d(i, j) = \frac{\sum_{f=1}^p \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^p \delta_{ij}^{(f)}};$$

where the indicator $\delta_{ij}^{(f)} = 0$ if either

- x_{if} or x_{jf} is missing (i.e., there is no measurement of variable f for object i or object j), or $x_{if} = x_{jf} = 0$ and variable f is asymmetric binary;
- otherwise, $\delta_{ij}^{(f)} = 1$

Variables of Mixed Types

- The contribution of variable f to the dissimilarity between i and j , that is, $d_{ij}^{(f)}$, is computed dependent on its type:
- If ' f ' is interval-based:
$$d_{ij}^{(f)} = \frac{|x_{if} - x_{jf}|}{\max_h x_{hf} - \min_h x_{hf}},$$

where h runs overall non missing objects for variable f

- If ' f ' is binary or categorical: $d_{ij}^{(f)} = 0$, if $x_{if} = x_{jf}$
 - otherwise $d_{ij}^{(f)} = 1$

Variables of Mixed Types

- If 'f' is ordinal: compute the ranks r_{if} and $z_{if} = (r_{if} - 1) / (M_f - 1)$, and treat z_{if} as interval scaled
- If 'f' is ratio-scaled: either perform logarithmic transformation and treat the transformed data as interval-scaled; or treat 'f' as continuous ordinal data, compute r_{if} and z_{if} , and then treat z_{if} as interval-scaled
- The above steps are identical to what we have already seen for each of the individual variable types

Variables of Mixed Types

- The only difference is for interval-based variables, where here we normalize so that the values map to the interval $[0.0, 1.0]$
- Thus, the dissimilarity between objects can be computed even when the variables describing the objects are of different types

Thank you





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Cluster analysis: Part - V

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Dissimilarity matrix for mixed type variables
- Python demo for computing different types of distances
- Python demo for computing distance matrix for interval scaled data



Example

Consider the data given in the following table and compute a dissimilarity matrix for the objects of the table

Now we will consider all of the variables, which are of different types

object identifier	test-1 (categorical)	test-2 (ordinal)	test-3 (ratio-scaled)
1	code-A	excellent	445
2	code-B	fair	22
3	code-C	good	164
4	code-A	excellent	1,210

Example

- The procedures we followed for test-1 (which is categorical) and test-2 (which is ordinal) are the same as outlined above for processing variables of mixed types
- For categorical variable - $d(i, j) = \frac{p - m}{p}$,
- For ordinal variable - $z_{if} = \frac{r_{if} - 1}{M_f - 1}$,
- For interval scale variable - $d_{ij}^{(f)} = \frac{|x_{if} - x_{jf}|}{\max_h x_{hf} - \min_h x_{hf}}$, ✓

Normalizing the interval scale data

- First, however, we need to complete some work for test-3 (which is ratio-scaled)
- We have already applied a logarithmic transformation to its values
- Based on the transformed values of 2.65, 1.34, 2.21, and 3.08 obtained for the objects 1 to 4, respectively, we let $\max_h x_h = 3.08$ and $\min_h x_h = \underline{1.34}$
- We then normalize the values in the dissimilarity matrix obtained in **Example solve for ratio data** by dividing each one by $(3.08 - 1.34) = 1.74$

Dissimilarity matrix for test-3

- This results in the following dissimilarity matrix for test-3:

Object Identifier	Ratio scaled Data (x)	Log (x)
1	445	2.65
2	22	1.34
3	164	2.21
4	1210	3.08



$$\begin{bmatrix} 0 & & & \\ \underline{0.75} & 0 & & \\ 0.25 & 0.50 & 0 & \\ \underline{0.25} & 1.00 & 0.50 & 0 \end{bmatrix}$$

- For 1 and 2 = $(2.65 - 1.34) / (3.08 - 1.34) = \underline{0.75}$

dissimilarity matrices for the three variables

- We can now use the dissimilarity matrices for the three variables in our computation of Equation $d_{ij}^{(f)} = \frac{|x_{if} - x_{jf}|}{\max_h x_{hf} - \min_h x_{hf}}$,
- For example, we get $d(2,1) = (1(1) + 1(1) + 1(0.75)) / 3 = 0.92$

$$\begin{bmatrix} 0 & & & & \\ \boxed{1} & 0 & & & \\ & 1 & 1 & 0 & \\ & \boxed{0} & 1 & 1 & 0 \end{bmatrix}$$

Dissimilarity matrix
for categorical

$$\begin{bmatrix} 0 & & & & \\ \boxed{1} & 0 & & & \\ & 0.5 & 0.5 & 0 & \\ & \boxed{0} & 1.0 & 0.5 & 0 \end{bmatrix}$$

Dissimilarity matrix
for ordinal

$$\begin{bmatrix} 0 & & & & \\ \boxed{0.75} & 0 & & & \\ & 0.25 & 0.50 & 0 & \\ & \boxed{0.25} & 1.00 & 0.50 & 0 \end{bmatrix}$$

normalize the values in the
dissimilarity matrix for ratio data

Example

- The resulting dissimilarity matrix obtained for the data described by the three variables of mixed types is:

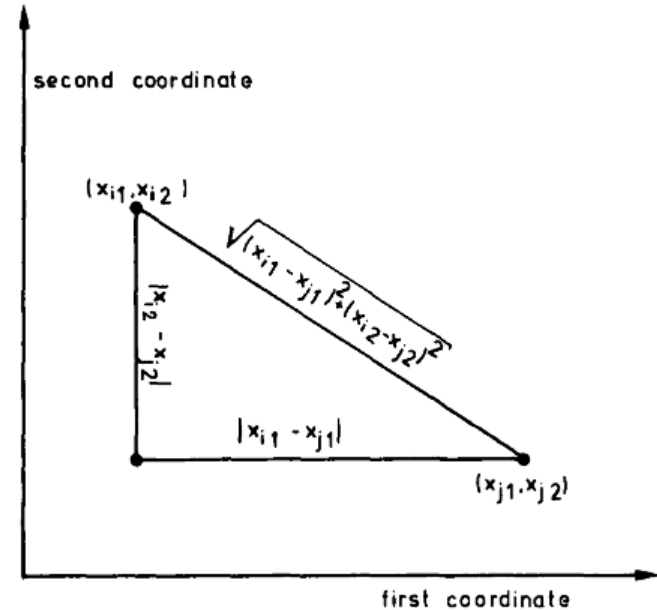
$$\begin{matrix} & & & & \\ & & & & \\ (2,1) & & & & \\ & & & & \\ (4,1) & & & & \end{matrix} \begin{bmatrix} 0 & & & & \\ 0.92 & 0 & & & \\ 0.58 & 0.67 & 0 & & \\ 0.08 & 1.00 & 0.67 & 0 & \end{bmatrix}$$

Interpretation

- If we go back and look at Table of given data, we can intuitively guess that objects 1 and 4 are the most similar, based on their values for test-1 and test-2
- This is confirmed by the dissimilarity matrix, where $d(4,1)$ is the lowest value for any pair of different objects
- Similarly, the matrix indicates that objects 2 and 4 are the least similar

Distance Measurement using python - Euclidean Distance :

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ip} - x_{jp})^2}$$



Python Demo for Euclidean Distance

```
In [1]: import scipy  
        from scipy.spatial import distance
```

#Euclidean Distance

```
In [2]: import numpy as np  
        a = [1,2,3]  
        b = [4,5,6]  
        dst = distance.euclidean(a,b)
```

```
In [3]: dst
```

```
Out[3]: 5.196152422706632
```

Distance Measurement using python – Minkowski Distance :

$$d(i, j) = (|x_{i1} - x_{j1}|^p + |x_{i2} - x_{j2}|^p + \cdots + |x_{in} - x_{jn}|^p)^{1/p},$$

- $p = 1$ Manhattan distance
- $p = 2$ Euclidean distance



Python Demo for Minkowski Distance

#Minkowski Distance

```
In [4]: distance.minkowski([1, 0, 0], [0, 1, 0], 1) #Manhattan distance
```

```
Out[4]: 2.0
```

```
In [5]: distance.minkowski([1, 0, 0], [0, 1, 0], 2) #Euclidean distance
```

```
Out[5]: 1.4142135623730951
```

```
In [6]: distance.minkowski([1, 2, 3], [4, 5, 6], 2) ✓
```

```
Out[6]: 5.196152422706632
```

```
In [7]: distance.minkowski([1, 2, 3], [4, 5, 6], 3) ✓
```

```
Out[7]: 4.3267487109222245
```

Dissimilarity matrix

#dissimilarity or distance matrix

```
In [9]: import pandas as pd
        from scipy.spatial import distance_matrix

        data = [[1, 4], [2, 5], [3, 6]]
        df = pd.DataFrame(data, columns=['a', 'b'])
        df
```

Out[9]:

	a	b
0	1	4
1	2	5
2	3	6

```
In [10]: pd.DataFrame(distance_matrix(df.values, df.values))
```

Out[10]:

	0	1	2
0	0.000000	1.414214	2.828427
1	1.414214	0.000000	1.414214
2	2.828427	1.414214	0.000000

Distance matrix calculation for Interval-Scaled Variables

- For example :
- Take eight people, the weight (in kilograms) and the height (in centimetres)
- In this situation, $n = 8$ and $p = 2$.

Person	Weight(Kg)	Height(cm)
A	15	95
B	49	156
C	13	95
D	45	160
E	85	178
F	66	176
G	12	90
H	10	78

#data matrix

```
In [5]: import pandas as pd
        from scipy.spatial import distance_matrix

        data = [[15, 95], [49, 156], [13, 95], [45, 160], [85, 178], [66, 176], [12, 90], [10, 78]]
        ctys = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
        df = pd.DataFrame(data, columns=['Weight', 'Height'], index=ctys)
```

```
In [6]: df
```

Out[6]:

	Weight	Height
A	15	95
B	49	156
C	13	95
D	45	160
E	85	178
F	66	176
G	12	90
H	10	78



```
In [7]: Distance_matrix = pd.DataFrame(distance_matrix(df.values, df.values), index=df.index, columns=df.index)
Distance_matrix
```

Out[7]:

	A	B	C	D	E	F	G	H
A	0.000000	69.835521	2.000000	71.589105	108.577162	95.718337	5.830952	17.720045
B	69.835521	0.000000	70.830784	5.656854	42.190046	26.248809	75.663730	87.206651
C	2.000000	70.830784	0.000000	72.449983	109.877204	96.798760	5.099020	17.262677
D	71.589105	5.656854	72.449983	0.000000	43.863424	26.400758	77.388630	89.157165
E	108.577162	42.190046	109.877204	43.863424	0.000000	19.104973	114.337221	125.000000
F	95.718337	26.248809	96.798760	26.400758	19.104973	0.000000	101.548018	112.871608
G	5.830952	75.663730	5.099020	77.388630	114.337221	101.548018	0.000000	12.165525
H	17.720045	87.206651	17.262677	89.157165	125.000000	112.871608	12.165525	0.000000

Distance matrix calculation using Python

```
In [8]: Distance_matrix.round(decimals=1, out=None)
```

Out[8]:

	A	B	C	D	E	F	G	H
A	0.0	69.8	2.0	71.6	108.6	95.7	5.8	17.7
B	69.8	0.0	70.8	5.7	42.2	26.2	75.7	87.2
C	2.0	70.8	0.0	72.4	109.9	96.8	5.1	17.3
D	71.6	5.7	72.4	0.0	43.9	26.4	77.4	89.2
E	108.6	42.2	109.9	43.9	0.0	19.1	114.3	125.0
F	95.7	26.2	96.8	26.4	19.1	0.0	101.5	112.9
G	5.8	75.7	5.1	77.4	114.3	101.5	0.0	12.2
H	17.7	87.2	17.3	89.2	125.0	112.9	12.2	0.0

Thank You





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

K- Means Clustering

Dr. A. Ramesh

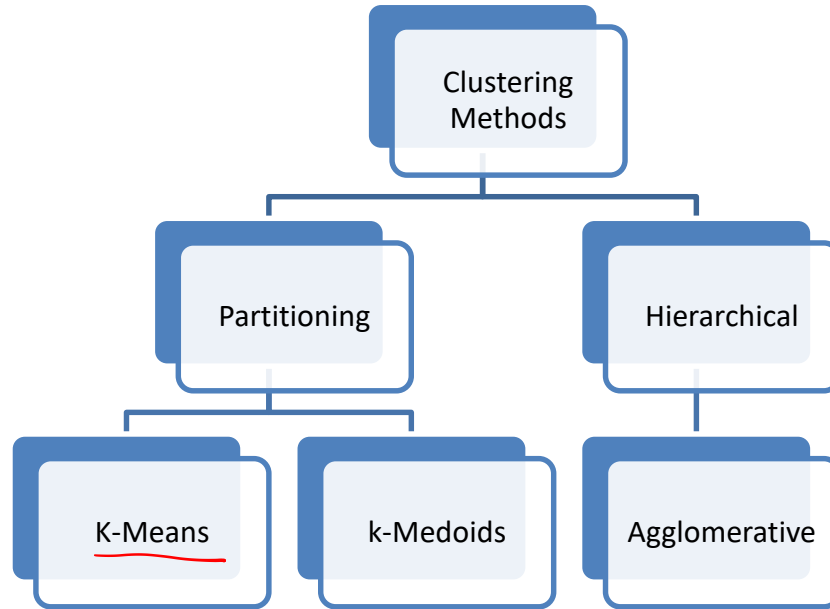
DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Classification of clustering methods
- Partitioning method: K – means clustering

Classification of Clustering Methods



Which Clustering Algorithm to Choose

- The choice of a clustering algorithm depends on
 - Type of data available
 - Particular purpose
- It is permissible to try several algorithms on the same data, because cluster analysis is mostly used as a descriptive or exploratory tool

Partitioning Method

Given -

- a data set of n objects
- k , the number of clusters
- A partitioning algorithm organizes the objects into k partitions ($k \leq n$), where each partition represents a cluster.
- The clusters are formed to optimize an objective partitioning criterion
- Objective partitioning criterion such as a dissimilarity function based on distance
- Therefore, the objects within a cluster are “similar,” whereas the objects of different clusters are “dissimilar” in terms of the data set attributes.

Partitioning Method

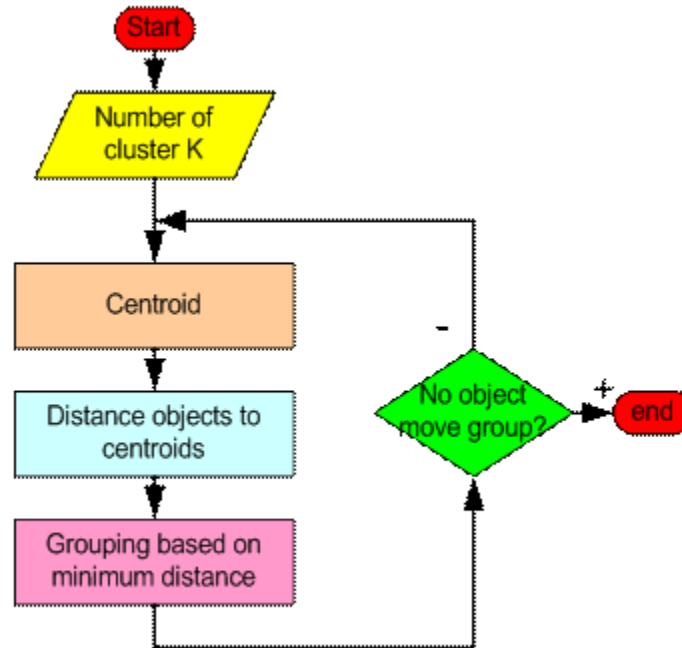
- Partitioning methods are applied if one wants to classify the objects into k clusters, where k is fixed.

K-Means Method

- It is a centroid based technique
- The k -means algorithm takes the input parameter, k , and partitions a set of n objects into k clusters
- So that the resulting intra-cluster similarity is high but the inter-cluster similarity is low
- Cluster similarity is measured in regard to the *mean* value of the objects in a cluster, which can be viewed as the cluster's centroid or center of gravity



Working Principle of K-Means Algorithm



Working Principle of K-Means Algorithm

- First it randomly selects k of the objects, each of which initially represents a cluster mean or center
- For each of the remaining objects, an object is assigned to the cluster to which it is the most similar, based on the distance between the object and the cluster mean
- It then computes the new mean for each cluster
- This process iterates until the criterion function converges

Working Principle of K-Means Algorithm

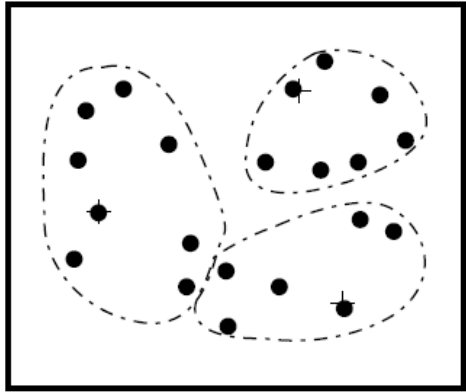
- Criterion function

$$E = \sum_{i=1}^k \sum_{p \in C_i} |p - m_i|^2$$

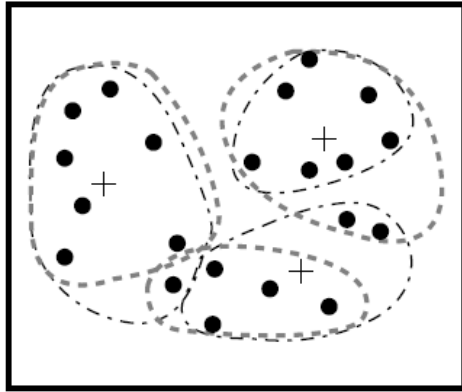
where

- E is the sum of the square error for all objects in the data set;
- p is the point in space representing a given object;
- m_i is the mean of cluster C_i (both p and m_i are multidimensional).
- For each object in each cluster, the distance from the object to its cluster center is squared, and the distances are summed.
- This criterion tries to make the resulting k clusters as compact and as separate as possible.

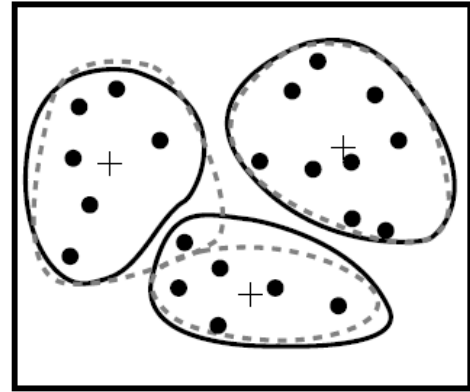
$$K = 3$$



(a)



(b)



(c)

K-Means Clustering Algorithm

Algorithm: k-means. The k-means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

- Input:
 - k: the number of clusters,
 - D: a data set containing n objects.
- Output: A set of k clusters.

K-Means Clustering Method

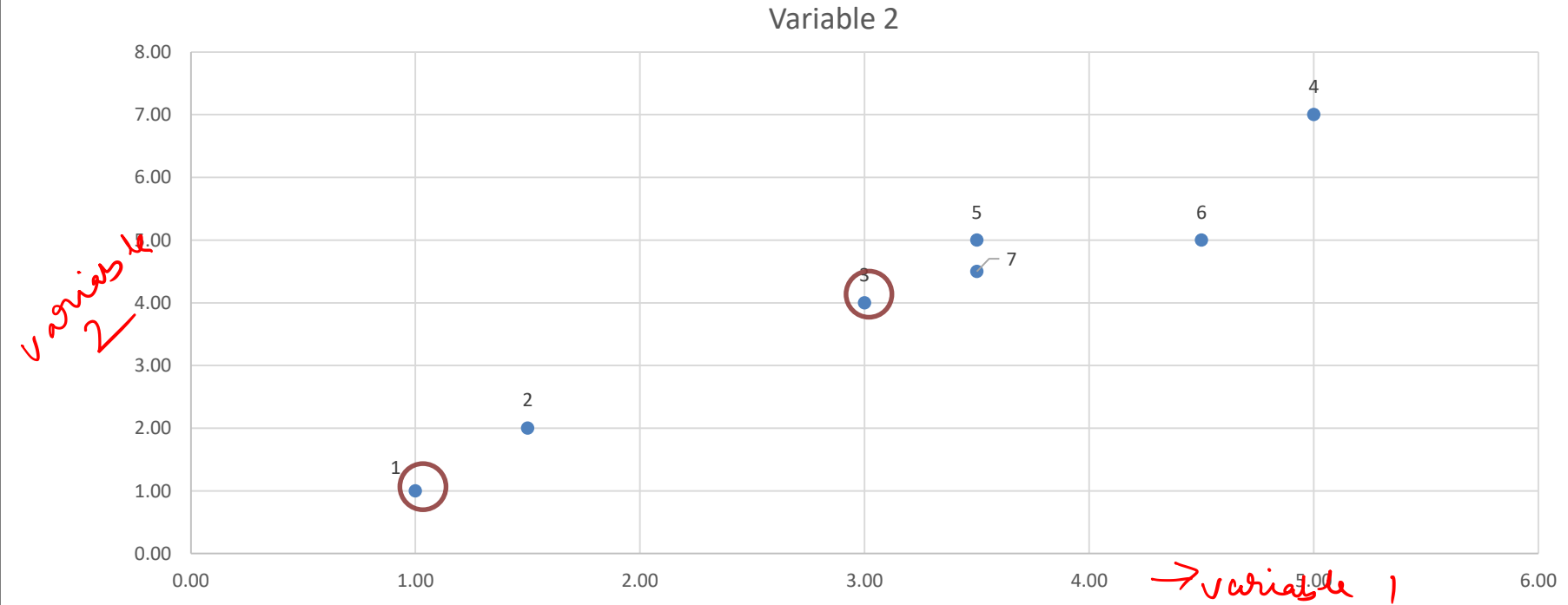
- Method:
 - (1) arbitrarily choose k objects from D as the initial cluster centers;
 - (2) repeat
 - (3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
 - (4) update the cluster means, i.e., calculate the mean value of the objects for each cluster;
 - (5) until no change;

K-Means clustering example

$K = 2$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

K-Means clustering example



K-Means clustering example

- Initialization: Randomly we choose following two centroids (k=2) for two clusters. In this case the 2 centroid are:

Cluster	Var1	Var2
K1	1.0	1.0
K2	3.0	4.0

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

- Calculate Euclidean distance using the given equation

$$\text{Distance} [(x_1, y_1), (x_2, y_2)] = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means clustering example

Distance of k1 from k1 (1.0, 1.0) = $\sqrt{(1.0 - 1.0)^2 + (1.0 - 1.0)^2} = 0$

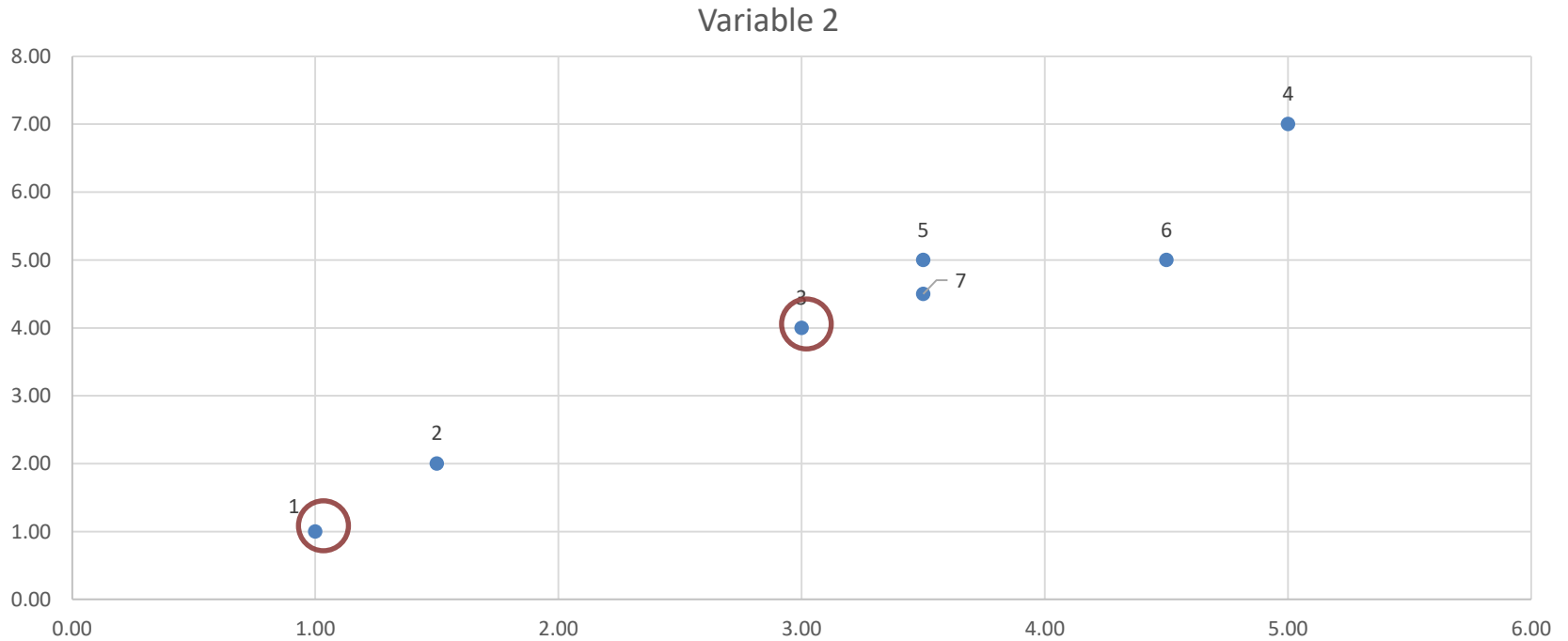
k1 to k2 (1.0, 1.0), (3.0, 4.0) = $\sqrt{(3.0 - 1.0)^2 + (4.0 - 1.0)^2} = 3.61$

Distance of k 2 from k2 (3.0, 4.0) = $\sqrt{(3.0 - 3.0)^2 + (4.0 - 4.0)^2} = 0$

Cluster	Centroid		
	K1	K2	Assignment
K1	0	3.61	k1
K2	3.61	0	k2

Indivi dual	Variab le 1	Variab le 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

At $K = 2$



K-Means clustering example

- Calculate Euclidean distance for next dataset (1.5, 2.0)

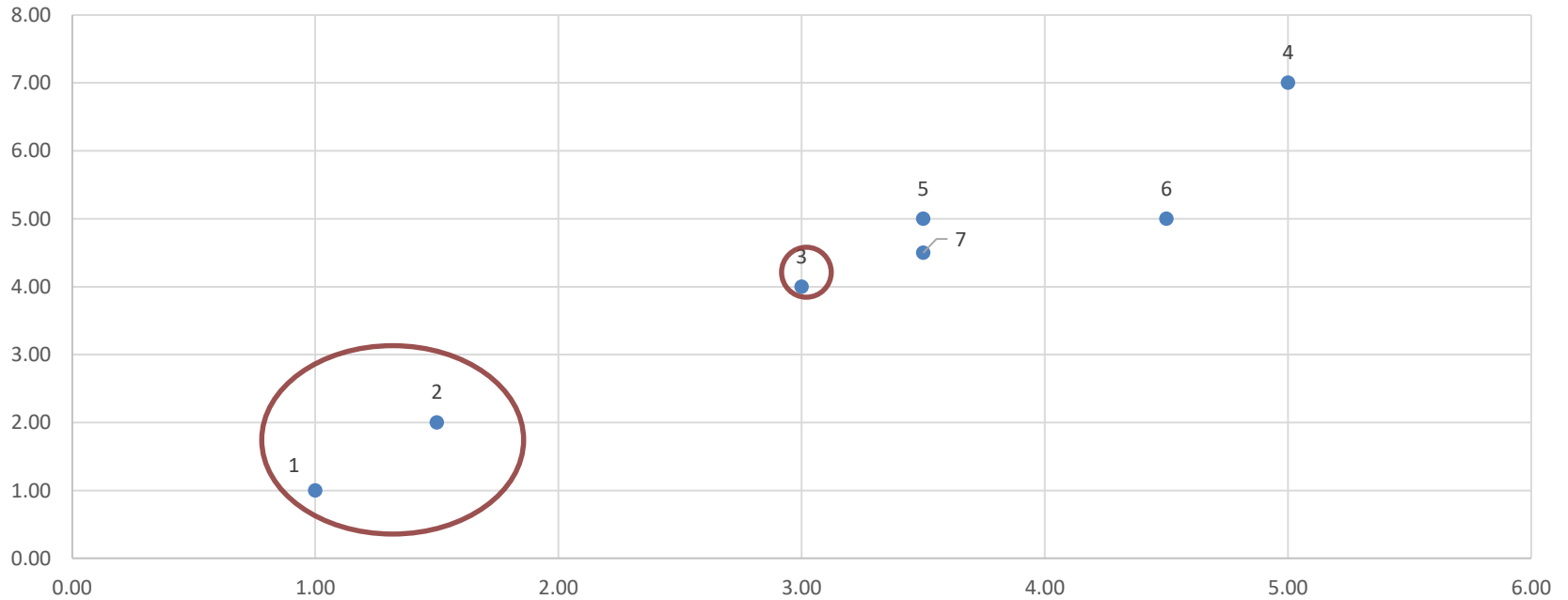
$$\text{Distance from cluster1} = \sqrt{(1.5 - 1.0)^2 + (2.0 - 1.0)^2} = 1.12$$

$$\text{Distance from cluster2} = \sqrt{(1.5 - 3.0)^2 + (2.0 - 4.0)^2} = 2.5$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Dataset	Euclidean Distance		
	Cluster 1	Cluster 2	Assignment
(1.5, 2.0)	1.12	2.5	k1

Variable 2



K-Means clustering example

- Update the cluster centroid

Cluster	Var1	Var2
K1	$(1.0 + 1.5)/2 = 1.25$	$(1.0 + 2.0)/2 = 1.5$
K2	3.0	4.0

K-Means clustering example

- Calculate Euclidean distance for next dataset (5.0, 7.0)

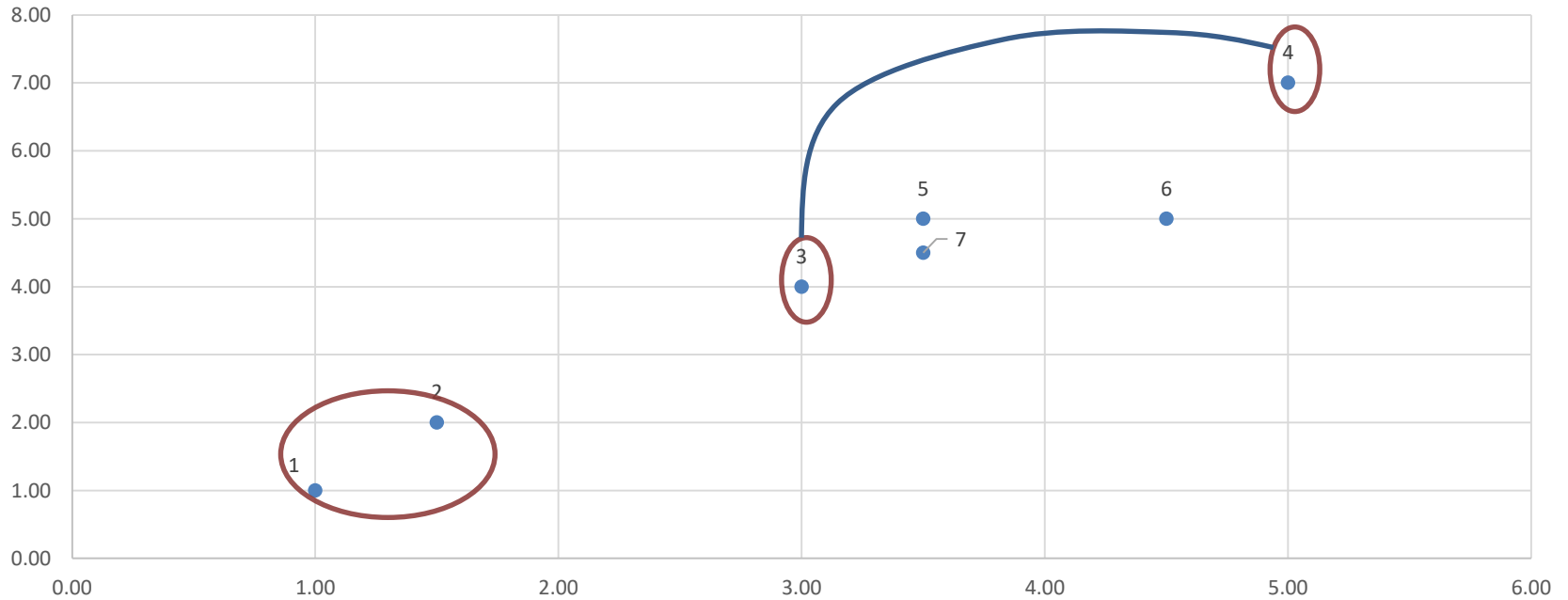
$$\text{Distance from cluster1} = \sqrt{(5.0 - 1.25)^2 + (7.0 - 1.5)^2} = 6.66$$

$$\text{Distance from cluster2} = \sqrt{(5.0 - 3.0)^2 + (7.0 - 4.0)^2} = 3.61$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Dataset	Euclidean Distance		
	Cluster 1	Cluster 2	Assignment
(5.0, 7.0)	6.66	3.61	K-2

Variable 2



K-Means clustering example

- Update the cluster centroid

Cluster	Var1	Var2
K1	1.25	1.5
K2	$(3.0 + 5.0)/2 = 4$	$(4.0 + 7.0)/2 = 5.5$

K-Means clustering example

- Calculate Euclidean distance for next dataset (3.5, 5.0)

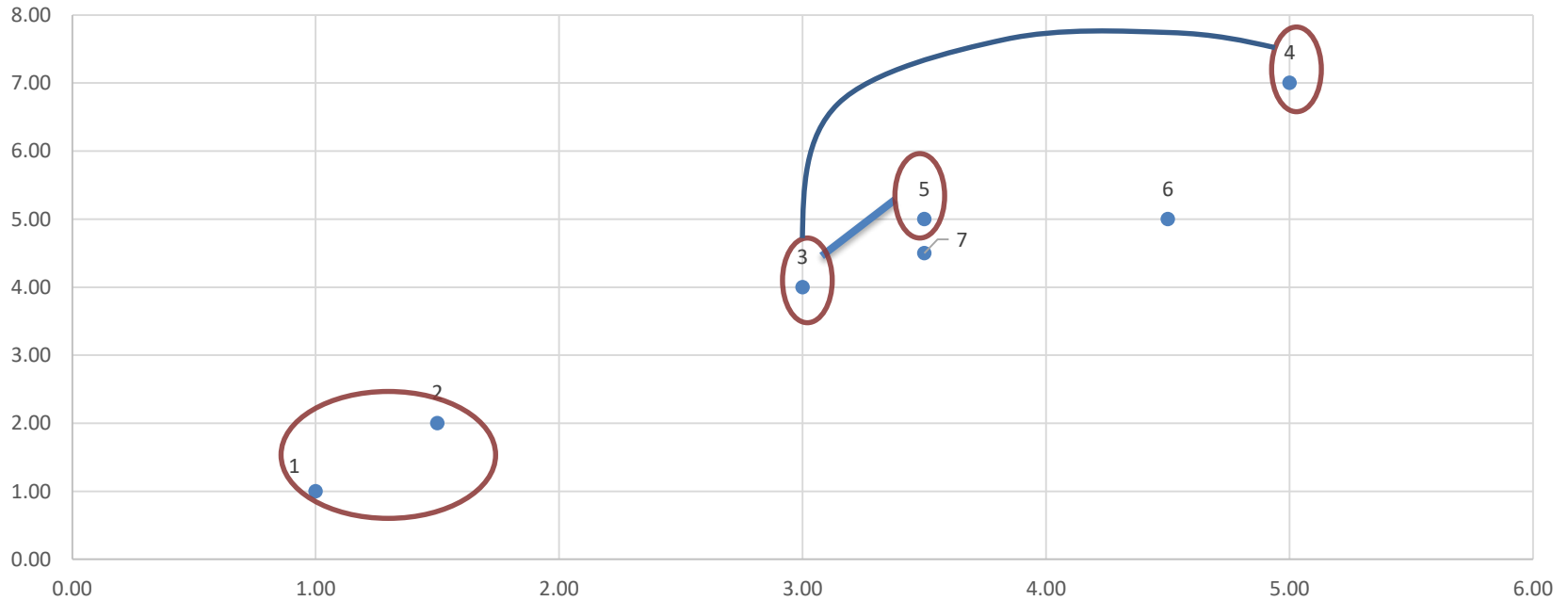
$$\text{Distance from cluster1} = \sqrt{(3.5 - 1.25)^2 + (5.0 - 1.5)^2} = 4.16$$

$$\text{Distance from cluster2} = \sqrt{(3.5 - 4.0)^2 + (5.0 - 5.5)^2} = 0.71$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Dataset	Euclidean Distance		
	Cluster 1	Cluster 2	Assignment
(3.5, 5.0)	4.16	0.71	K-2

Variable 2



K-Means clustering example

- Update the cluster centroid

Cluster	Var1	Var2
K1	1.25	1.5
K2	$(3.0+5.0+ 3.5)/3 =$ 3.83 /	$(4.0+7.0 + 5.0)/3 =$ 5.33 /

K-Means clustering example

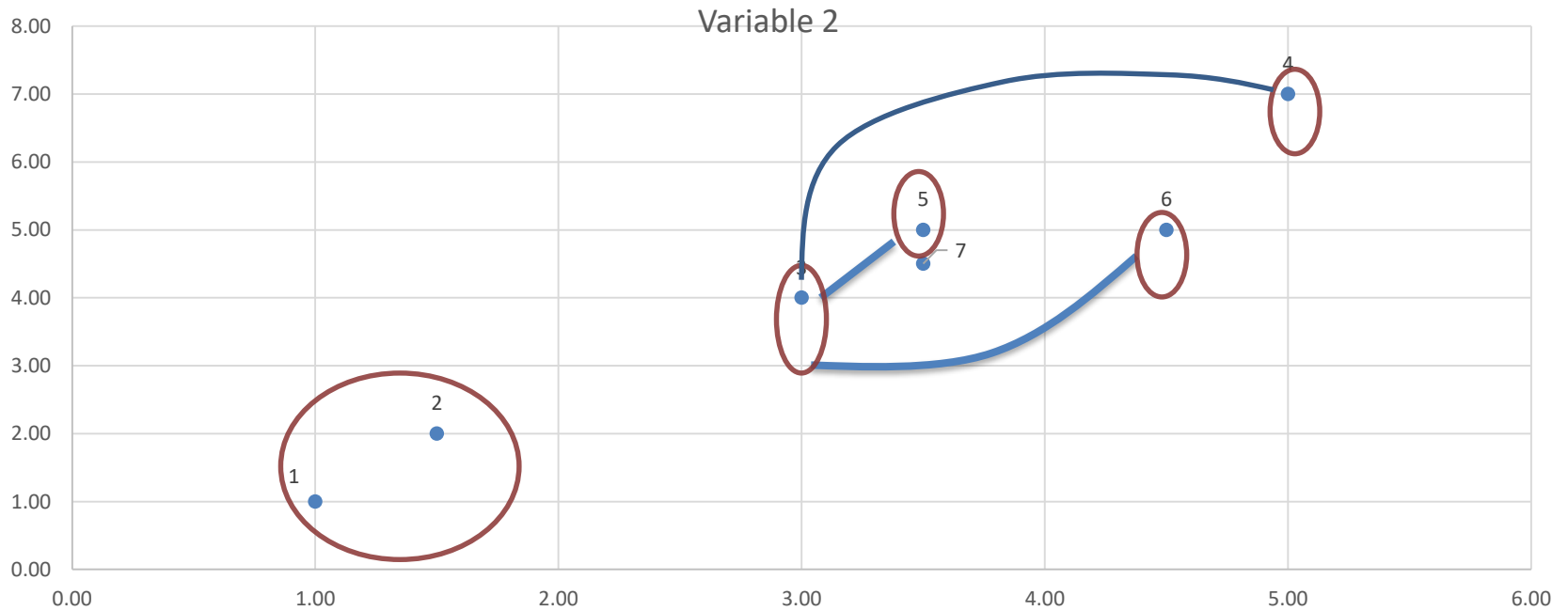
- Calculate Euclidean distance for next dataset (4.5, 5.0)

$$\text{Distance from cluster1} = \sqrt{(4.5 - 1.25)^2 + (5.0 - 1.5)^2} = 4.78$$

$$\text{Distance from cluster2} = \sqrt{(4.5 - 3.83)^2 + (5.0 - 5.33)^2} = 0.75$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Dataset	Euclidean Distance		
	Cluster 1	Cluster 2	Assignment
(4.5, 5.0)	4.78	<u>0.75</u>	K- 2



K-Means clustering example

- Update the cluster centroid

Cluster	Var1	Var2
K1	1.25	1.5
K2	$(3.0+5.0+3.5+4.5)/4= 4.00$	$(4.0+7.0+5.0+5.0)/4= 5.25$

K-Means clustering example

- Calculate Euclidean distance for next dataset (3.5, 4.5)

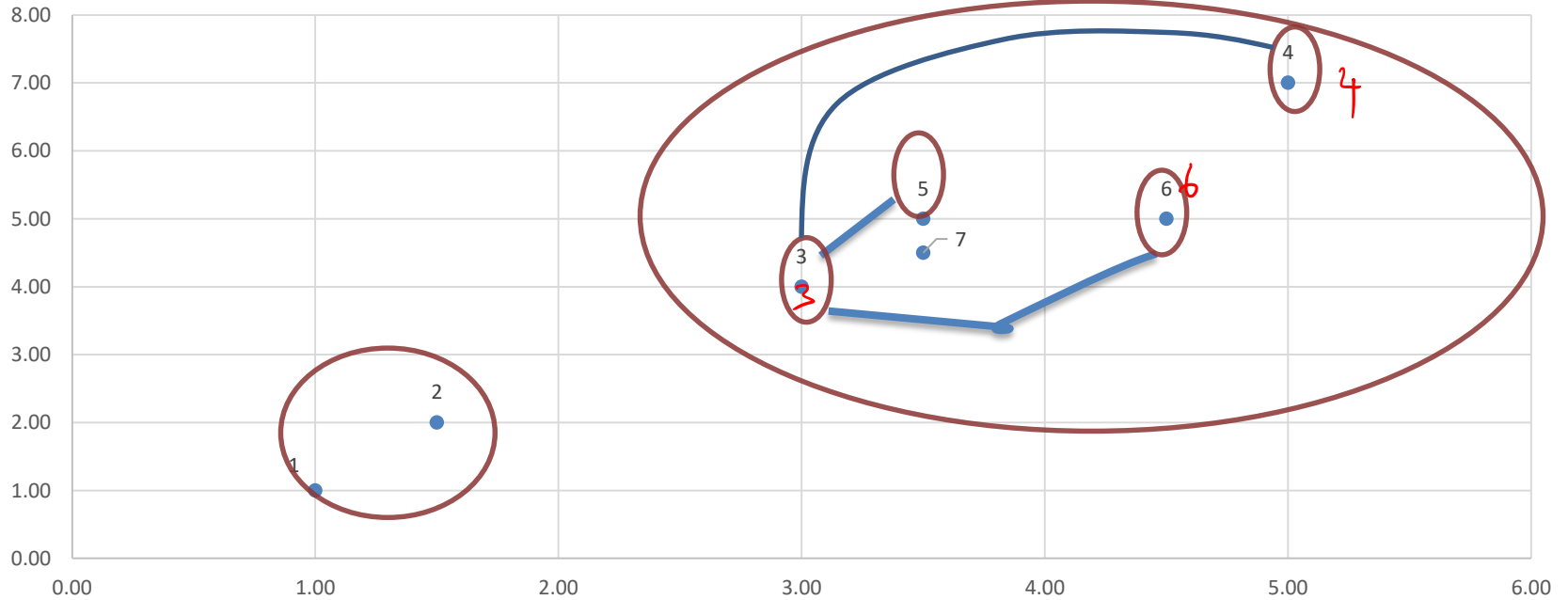
$$\text{Distance from cluster1} = \sqrt{(3.5 - 1.25)^2 + (4.5 - 1.5)^2} = 3.75$$

$$\text{Distance from cluster2} = \sqrt{(3.5 - 4.00)^2 + (4.5 - 5.25)^2} = 0.86$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Dataset	Euclidean Distance		
	Cluster 1	Cluster 2	Assignment
(3.5, 4.5)	3.75	<u>0.86</u>	K-2

Variable 2



K-Means clustering example

- Update the cluster centroid

Cluster	Var1	Var2
K1	1.25	1.5
K2	$(3.0+5.0+3.5+4.5+3.5)/5 = 3.9$	$(4.0+7.0+5.0+5.0+4.5)/5 = 5.1$

K-Means clustering example

Individual	Variable 1	Variable 2	Assignment
1	1.0	1.0	1
2	1.5	2.0	1
3	3.0	4.0	2
4	5.0	7.0	2
5	3.5	5.0	2
6	4.5	5.0	2
7	3.5	4.5	2

Python code for K- Means Clustering

```
In [1]: import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```
In [2]: data = pd.read_excel('clustering_ex.xlsx')
```

```
In [3]: data
```

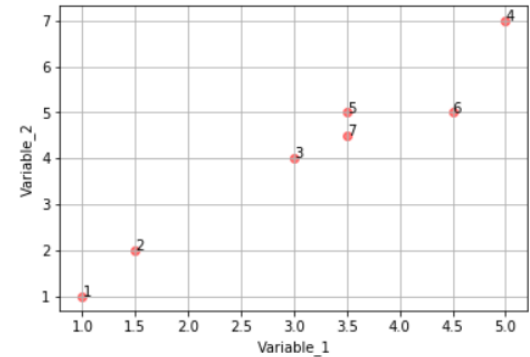
Out[3]:

	Variable_1	Variable_2
0	1.0	1.0
1	1.5	2.0
2	3.0	4.0
3	5.0	7.0
4	3.5	5.0
5	4.5	5.0
6	3.5	4.5

Python code for K- Means Clustering

```
In [4]: fig = plt.figure(figsize = (5, 5))
x = data['Variable_1']
y = data['Variable_2']
n = range(1,8)
fig, ax = plt.subplots()
ax.scatter(x, y, marker='o', c='red', alpha=0.5)
plt.grid()
plt.xlabel("Variable_1")
plt.ylabel("Variable_2")
for i, txt in enumerate(n):
    ax.annotate(txt, (x[i], y[i]))
```

<matplotlib.figure.Figure at 0x20d7a5044a8>



Python code

```
In [5]: from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=2)
kmeans.fit(data)
```

```
Out[5]: KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
               n_clusters=2, n_init=10, n_jobs=None, precompute_distances='auto',
               random_state=None, tol=0.0001, verbose=0)
```

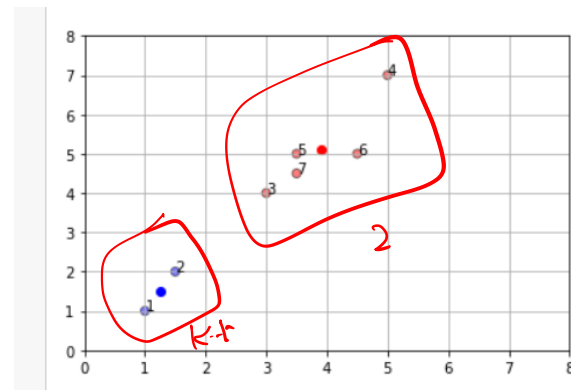
```
In [6]: labels = kmeans.predict(data)
        centroids = kmeans.cluster_centers_
```

```
In [8]: centroids
```

```
Out[8]: array([[3.9 , 5.1 ],
               [1.25, 1.5 ]])
```

```
In [9]: fig = plt.figure(figsize = (5, 5))
        colormap = {1:'r', 2:'b'}
        colors = map(lambda x: colormap[x+1], labels)
        colors1 = list(colors)
        fig, ax = plt.subplots()
        ax.scatter(x, y, color = colors1, alpha = 0.5, edgecolor = 'k')
        for idx, centroid in enumerate(centroids):
            plt.scatter(*centroid, color = colormap[idx+1])

        for i, txt in enumerate(n):
            ax.annotate(txt, (x[i], y[i]))
        plt.grid()
        plt.xlim(0, 8)
        plt.ylim(0, 8)
        plt.show()
```



Thank you





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Hierarchical method of clustering - I

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Introduction to Hierarchical clustering
- Partitioning Vs. Hierarchical

Introduction

- A hierarchical method creates a hierarchical decomposition of the given set of data objects
- A hierarchical clustering method works by grouping data objects into a tree of clusters
- A hierarchical method can be classified as being either agglomerative or divisive, based on how the hierarchical decomposition is formed
- The agglomerative approach, also called the bottom-up approach, starts with each object forming a separate group

Introduction

- It successively merges the objects or groups that are close to one another, until all of the groups are merged into one (the topmost level of the hierarchy), or until a termination condition holds
- The divisive approach, also called the top-down approach, starts with all of the objects in the same cluster
- In each successive iteration, a cluster is split up into smaller clusters, until eventually each object is in one cluster, or until a termination condition holds

Introduction

- Hierarchical methods suffer from the fact that once a step (merge or split) is done, it can never be undone
- This rigidity is useful in that it leads to smaller computation costs by not having to worry about a combinatorial number of different choices
- However, such techniques cannot correct erroneous decisions

Agglomerative and Divisive Hierarchical Clustering

Agglomerative



- This bottom-up strategy starts by placing each object in its own cluster and then merges these atomic clusters into larger and larger clusters, until all of the objects are in a single cluster or until certain termination conditions are satisfied
- Most hierarchical clustering methods belong to this category

Divisive Hierarchical



- This top-down strategy does the reverse of agglomerative hierarchical clustering by starting with all objects in one cluster
- It subdivides the cluster into smaller and smaller pieces, until each object forms a cluster on its own or until it satisfies certain termination conditions, such as a desired number of clusters is obtained or the diameter of each cluster is within a certain threshold

Agglomerative versus divisive hierarchical clustering

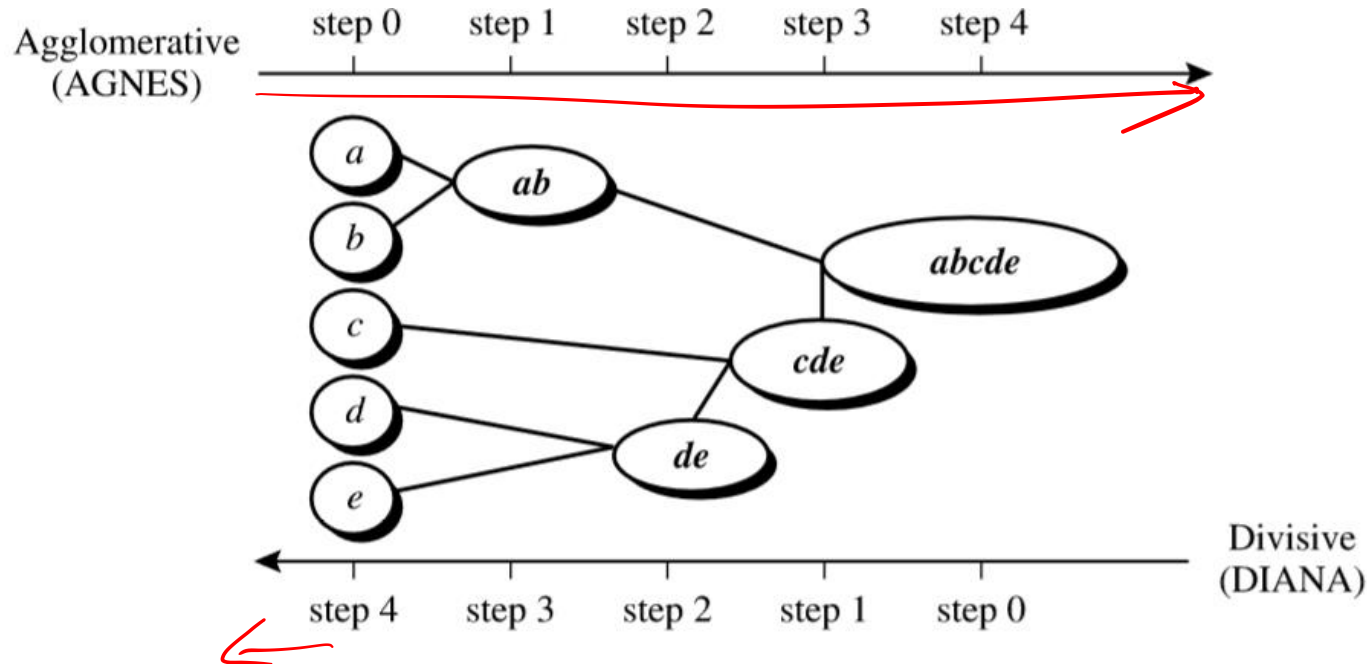


Figure: 1 Agglomerative and divisive hierarchical clustering on data objects $\{a, b, c, d, e\}$

Interpretation

- Figure: 1 shows the application of AGNES (AGglomerative NESTing), an agglomerative hierarchical clustering method, and DIANA (DIvisive ANAlysis), a divisive hierarchical clustering method, to a data set of five objects, {a,b,c,d,e}
- Initially, AGNES places each object into a cluster of its own
- The clusters are then merged step-by-step according to some criterion
- Let's say for example, clusters C_1 and C_2 may be merged if an object in C_1 and an object in C_2 form the minimum Euclidean distance between any two objects from different clusters

Interpretation

- This is a single-linkage approach in that each cluster is represented by all of the objects in the cluster, and the similarity between two clusters is measured by the similarity of the closest pair of data points belonging to different clusters
- The cluster merging process repeats until all of the objects are eventually merged to form one cluster

Interpretation

- In DIANA, all of the objects are used to form one initial cluster
- The cluster is split according to some principle, such as the maximum Euclidean distance between the closest neighboring objects in the cluster
- The cluster splitting process repeats until, eventually, each new cluster contains only a single object
- In either agglomerative or divisive hierarchical clustering, the user can specify the desired number of clusters as a termination condition

Dendrogram

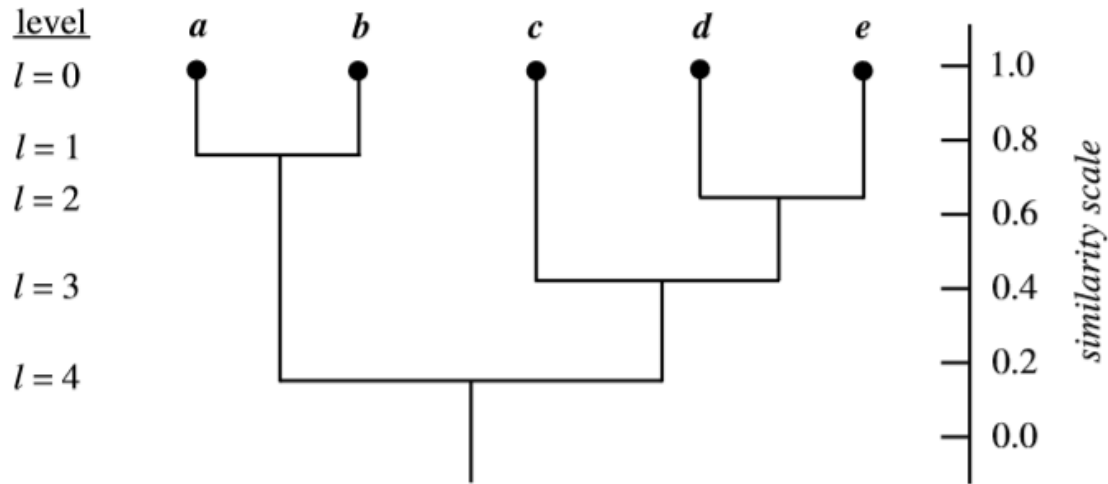


Figure 2: Dendrogram representation for hierarchical clustering of data objects $\{a, b, c, d, e\}$

Dendrogram

- A tree structure called a dendrogram is commonly used to represent the process of hierarchical clustering
- It shows how objects are grouped together step by step
- Figure: 2 shows a dendrogram for the five objects presented in Figure:1 , where $l = 0$ shows the five objects as singleton clusters at level 0
- At $l = 1$, objects a and b are grouped together to form the first cluster, and they stay together at all subsequent levels

Dendrogram

- We can also use a vertical axis to show the similarity scale between clusters
- For example, when the similarity of two groups of objects, $\{a,b\}$ and $\{c,d,e\}$, is roughly 0.16, they are merged together to form a single cluster

Measures for distance between clusters

- Four widely used measures for distance between clusters are as follows, where $|p-p'|$ is the distance between two objects or points, p and p' , m_i is the mean for cluster, C_i and n_i is the number of objects in C_i
- Minimum distance: $d_{\min}(C_i, C_j) = \min_{p \in C_i, p' \in C_j} |p-p'|$
- Maximum distance: $d_{\max}(C_i, C_j) = \max_{p \in C_i, p' \in C_j} |p-p'|$
- Mean distance: $d_{\text{mean}}(C_i, C_j) = |m_i - m_j|$
- Average distance: $d_{\text{avg}}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i} \sum_{p' \in C_j} |p - p'|$

Measures for distance between clusters

- When an algorithm uses the minimum distance, $d_{\min}(C_i, C_j)$, to measure the distance between clusters, it is sometimes called a nearest-neighbor clustering algorithm
- Moreover, if the clustering process is terminated when the distance between nearest clusters exceeds an arbitrary threshold, it is called a single-linkage algorithm
- If we view the data points as nodes of a graph, with edges forming a path between the nodes in a cluster, then the merging of two clusters, C_i and C_j , corresponds to adding an edge between the nearest pair of nodes in C_i and C_j

Measures for distance between clusters

- Because edges linking clusters always go between distinct clusters, the resulting graph will generate a tree
- Thus, an agglomerative hierarchical clustering algorithm that uses the minimum distance measure is also called a minimal spanning tree algorithm
- When an algorithm uses the maximum distance, $d_{\max}(C_i, C_j)$, to measure the distance between clusters, it is sometimes called a farthest-neighbor clustering algorithm
- If the clustering process is terminated when the maximum distance between nearest clusters exceeds an arbitrary threshold, it is called a complete-linkage algorithm

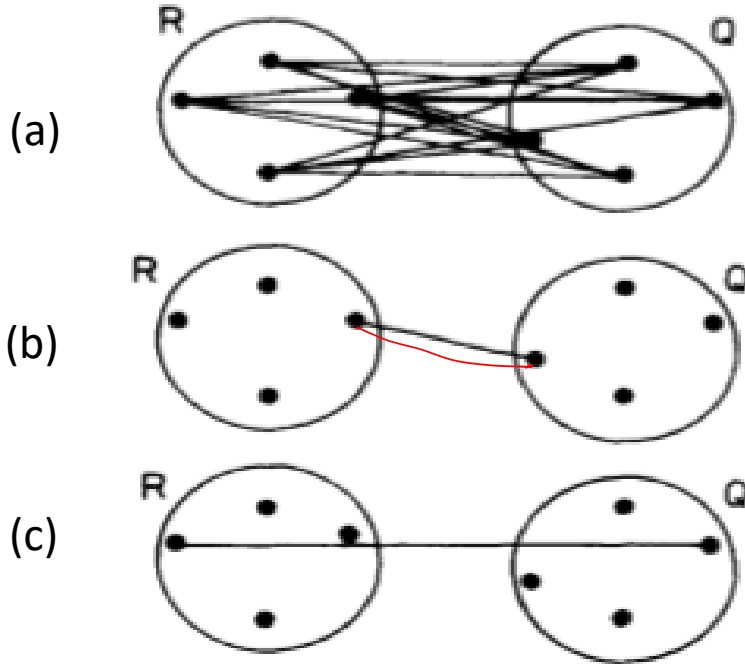
Measures for distance between clusters

- By viewing data points as nodes of a graph, with edges linking nodes, we can think of each cluster as a complete sub graph, that is, with edges connecting all of the nodes in the clusters
- The distance between two clusters is determined by the most distant nodes in the two clusters
- Farthest-neighbor algorithms tend to minimize the increase in diameter of the clusters at each iteration as little as possible
- If the true clusters are rather compact and approximately equal in size, the method will produce high-quality clusters
- Otherwise, the clusters produced can be meaningless

Choice of measurement

- The above minimum and maximum measures represent two extremes in measuring the distance between clusters
- They tend to be overly sensitive to outliers or noisy data
- The use of mean or average distance is a compromise between the minimum and maximum distances and overcomes the outlier sensitivity problem
- Whereas the mean distance is the simplest to compute, the average distance is advantageous in that it can handle categorical as well as numeric data

Illustration



Representation of some definitions of inter-cluster dissimilarity:

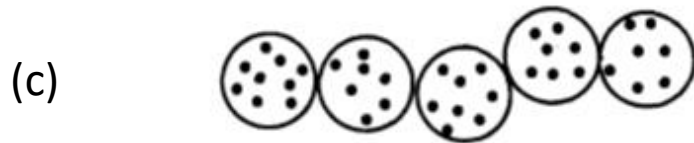
- (a) Group average
- (b) Nearest neighbor
- (c) Furthest neighbor

Illustration



Some types of clusters:

- (a) Ball-shaped
- (b) Elongated
- (c) Compact but not well separated



Difficulties with hierarchical clustering

- The hierarchical clustering method, though simple, often encounters difficulties regarding the selection of merge or split points
- Such a decision is critical because once a group of objects is merged or split, the process at the next step will operate on the newly generated clusters
- It will neither undo what was done previously nor perform object swapping between clusters

Difficulties with hierarchical clustering

- Thus merge or split decisions, if not well chosen at some step, may lead to low-quality clusters
- Moreover, the method does not scale well, because each decision to merge or split requires the examination and evaluation of a good number of objects or cluster
- For improving the clustering quality of hierarchical methods is to integrate hierarchical clustering with other clustering techniques, resulting in multiple-phase clustering

Partitioning Vs. Hierarchical

Method	General Characteristics
Partitioning methods	– Find mutually exclusive clusters of spherical shape – Distance-based – May use mean or medoid (etc.) to represent cluster center – Effective for small- to medium-size data sets
Hierarchical methods	– Clustering is a hierarchical decomposition (i.e., multiple levels) – Cannot correct erroneous merges or splits – May incorporate other techniques like microclustering or consider object “linkages”

K-means versus hierarchical clustering



K means versus hierarchical clustering

K- means clustering

- Non-hierarchical methods, such as k-means, using a pre-specified number of clusters, the method assigns records to each cluster to find the mutually exclusive cluster of spherical shape based on distance
- In this case, one can use mean or median as a cluster centre to represent each cluster

Hierarchical clustering

- Hierarchical methods can be either agglomerative or divisive
- Agglomerative methods begin with 'n' clusters and sequentially merge similar clusters until a single cluster is obtained

K means versus hierarchical clustering

K- means clustering

- These methods are generally less computationally intensive and are therefore preferred with very large datasets

Hierarchical clustering

- Divisive methods work in the opposite direction, starting with one cluster that includes all records
- Hierarchical methods are especially useful when the goal is to arrange the clusters into a natural hierarchy

K means versus hierarchical clustering

K- means clustering

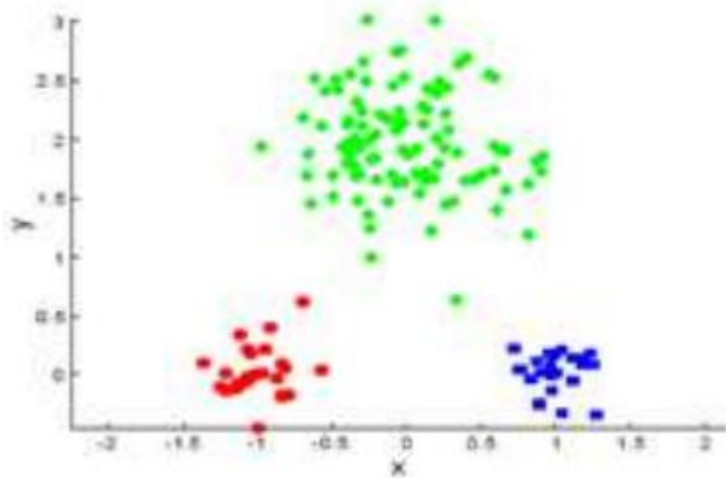
- A partitioning (K- means) clustering is simply a division of the set of data objects into non-overlapping subsets (clusters) such that each data object is in exactly one subset)

Hierarchical clustering

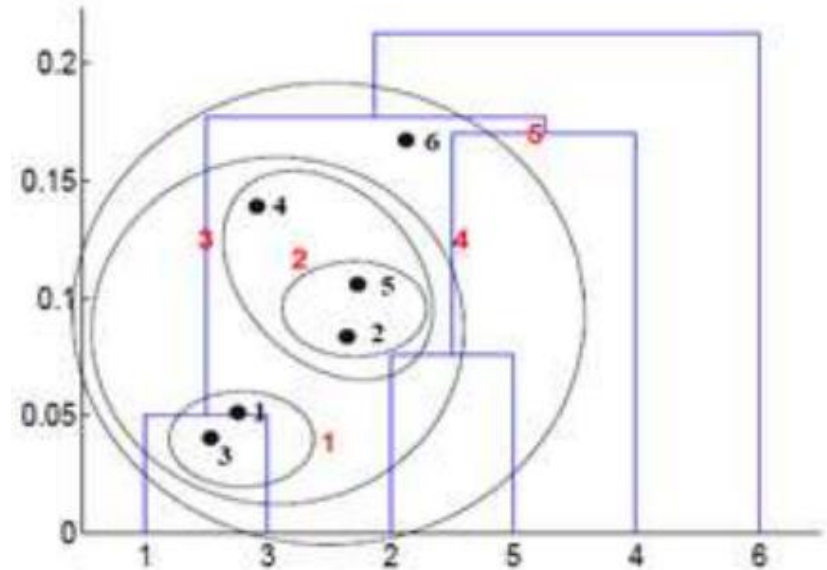
- A hierarchical clustering is a set of nested clusters that are organized as a tree

K means versus hierarchical clustering

Un-nested cluster



Nested cluster



Ashok, A.R., Prabhakar, C.R. and Dyaneshwar, P.A., Comparative Study on Hierarchical and Partitioning Data Mining Methods.

K means versus hierarchical clustering

- Hierarchical clustering does not assume a particular value of ' k ', as needed by k -means clustering
- The generated tree may correspond to a meaningful taxonomy
- Only a distance or “proximity” matrix is needed to compute the hierarchical clustering

	a	b	c	d	e	f
a	0	184	222	177	216	231
b	184	0	45	123	128	200
c	222	45	0	129	121	203
d	177	123	129	0	46	83
e	216	128	121	46	0	83
f	231	200	203	83	83	0

Proximity
matrix

K means versus hierarchical clustering

K Means clustering

- In K Means clustering, since one start with random choice of clusters, the results produced by running the algorithm multiple times might differ
- K Means is found to work well when the shape of the clusters is hyper spherical (like circle in 2D, sphere in 3D)

Hierarchical clustering

- Results are reproducible in Hierarchical clustering
- Hierarchical clustering don't work as well as, k means when the shape of the clusters is hyper spherical

K means versus hierarchical clustering

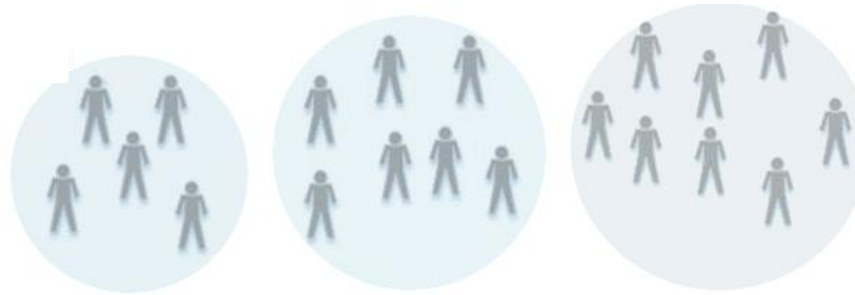
K Means clustering

- K Means clustering requires prior knowledge of K i.e. no. of clusters one want to divide your data into

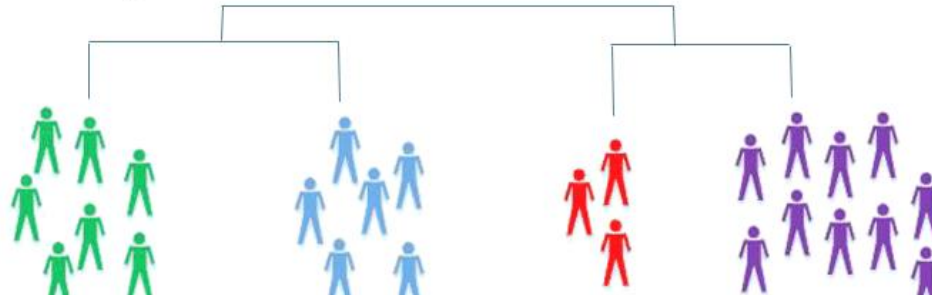
Hierarchical clustering

- In hierarchical clustering one can stop at any number of clusters, one find appropriate by interpreting the dendrogram

K means versus hierarchical clustering



Comparison of Kmeans and Hierarchical Clustering



<https://stepupanalytics.com/difference-between-k-means-clustering-and-hierarchical-clustering/>

Hierarchical clustering

Advantages

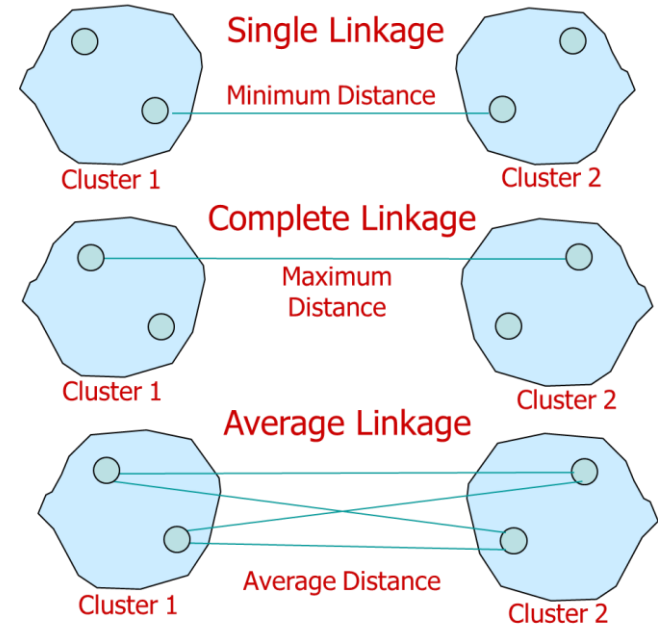
- Ease of handling of any forms of similarity or distance
- Consequently, applicability to any attributes types

Limitations of Hierarchical Clustering

- Hierarchical clustering requires the computation and storage of an $n \times n$ distance matrix. For very large datasets, this can be expensive and slow
- The hierarchical algorithm makes only one pass through the data. This means that records that are allocated incorrectly early in the process cannot be reallocated subsequently
- Hierarchical clustering also tends to have low stability. Reordering data or dropping a few records can lead to a different solution

Limitations of Hierarchical Clustering

- With respect to the choice of distance between clusters, single and complete linkage are robust to changes in the distance metric (e.g., Euclidean, statistical distance) as long as the relative ordering is kept.
- In contrast, average linkage is more influenced by the choice of distance metric, and might lead to completely different clusters when the metric is changed
- Hierarchical clustering is sensitive to outlier



Average-linkage clustering

- Compromise between Single and Complete Link
- Strengths
 - Less susceptible to noise and outliers
- Limitations
 - Biased towards globular clusters

Distance between two clusters

- **Ward's distance** between clusters C_i and C_j is the **difference** between the **total within cluster sum of squares for the two clusters separately**, and the **within cluster sum of squares resulting from merging the two clusters** in cluster C_{ij}

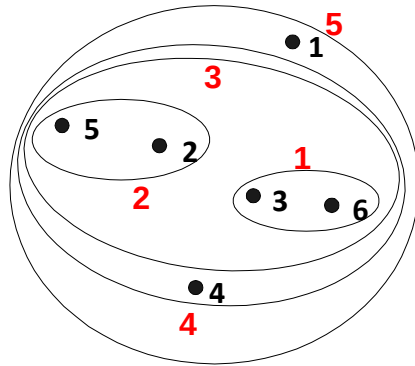
$$D_w(C_i, C_j) = \sum_{x \in C_i} (x - r_i)^2 + \sum_{x \in C_j} (x - r_j)^2 - \sum_{x \in C_{ij}} (x - r_{ij})^2$$

- r_i : centroid of C_i
- r_j : centroid of C_j
- r_{ij} : centroid of C_{ij}

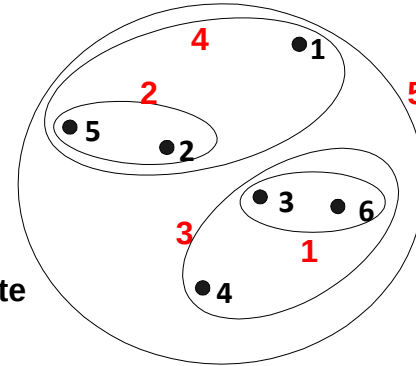
Ward's distance for clusters

- Similar to group average and centroid distance
- Less susceptible to noise and outliers
- Biased towards globular clusters
- Hierarchical analogue of k-means
 - Can be used to initialize k-means

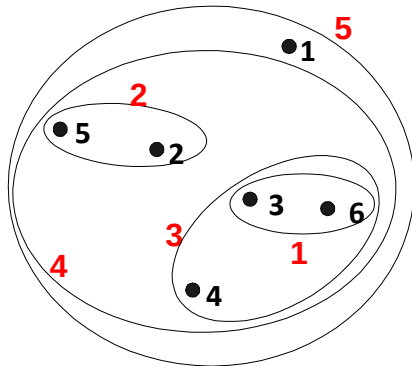
Hierarchical Clustering: Comparison



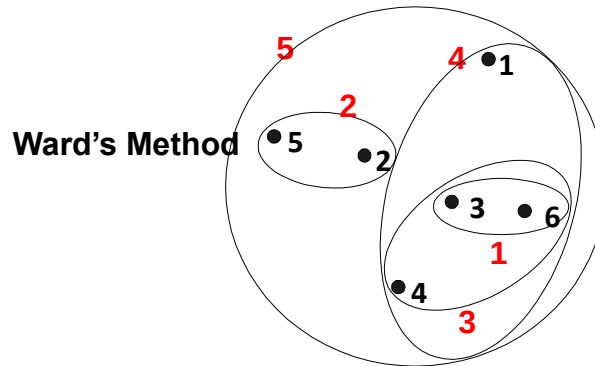
Simple linkage



Complete linkage



Group Average



Ward's Method

K- means clustering

Advantages

- The center of mass can be found efficiently by finding the mean value of each co-ordinate
- This leads to an efficient algorithm to compute the new centroids with a single scan of the data

Disadvantages

- K-means has problems when clusters are of differing sizes, densities, non-globular shapes and when the data contains outliers

Similarity

- Two most popular methods: hierarchical agglomerative clustering and k-means clustering
- In both cases, we need to define two types of distances: distance between two records and distance between two cluster
- In both cases, there is a variety of metrics that can be used

Thank You

